

# Energy-Aware LLM Inference on Edge Devices with Latency Constraints

**Abstract**—Deploying large language models (LLMs) on edge devices such as smartphones and tablets enables new applications but incurs substantial energy consumption and shortens battery life. Although Dynamic Voltage and Frequency Scaling (DVFS) can reduce energy use, the large strategy space arising from variable input and output sequence lengths is computationally prohibitive to explore. We introduce an analytical model that captures the relationship between inference latency, energy consumption, and hardware parameters. Based on this model, we design a lightweight DVFS strategy that dynamically adjusts hardware settings to minimize energy consumption while adhering to user-defined latency constraints. The framework is generalizable across different hardware platforms and requires minimal calibration. To evaluate the versatility of the framework, we conducted extensive experiments across various hardware and software platforms, including the Jetson AGX Orin, Raspberry Pi 4B, and Xiaomi Tablet. Under a 15% latency-relaxation budget, our framework reduces energy consumption by up to 26.78% for LLaMA-3-8B inference on GPU, with only a 6.40% latency increase. Our framework provides a practical and generalized solution for energy-efficient LLM deployment on diverse edge platforms.

## I. INTRODUCTION

The growing adoption of large language models (LLMs) for on-device inference enables applications such as personalized AI assistants, real-time language translation, and context-aware services without reliance on cloud connectivity. However, the high computational demands of LLMs lead to unsustainable energy drain, drastically shortening the operational lifespan of battery-powered devices, such as smartphones and tablets.

While sustainable AI is a well-recognized problem, existing studies mainly perform energy optimization from the perspective of a service provider (data center), where the service provider tries to balance between energy consumption and Service Level Objectives (SLOs) through job scheduling and cluster management [1], [2]. Existing studies on local LLM deployment mainly focus on reducing computational overhead through techniques such as pruning [3], [4] and quantization [5]–[7]. Although recent studies have explored energy-aware LLM inference [8], [9], they mainly target at specialized hardware/accelerators, which may not be compatible with general-purpose edge hardware.

Roofline analysis reveals that the prefill phase in LLM inference is predominantly compute-bound, whereas the decoding phase is constrained by memory bandwidth, highlighting distinct bottlenecks in each stage. It brings the feasibility that we can adopt dynamic voltage and frequency scaling (DVFS) strategy to dynamically adjust the frequency of computing

units (CPU/GPU), reducing idle processor cycles during memory access in the decoding phase. While DVFS is not a novel concept, under the context of LLM inference, how to choose the optimal strategy presents unique challenges. During autoregressive decoding, the computation workload is dependent on the sequence length. Thus, the possible strategy space is prohibitively large, considering the possible combinations of processor/memory working frequencies, input prompt length, and output token length.

To address these challenges, we propose a joint latency–energy modeling and optimization framework for autoregressive LLM inference on edge devices. To support heterogeneous CPU/GPU-based platforms, the framework abstracts away architecture-specific implementation details, such as SIMD instructions and CUDA kernels. Instead, it models inference latency and energy consumption using a small set of tunable hardware parameters, including the number of active compute units, compute frequency, and memory frequency. This hardware-agnostic abstraction makes the framework applicable to a broad class of general-purpose von Neumann architectures.

The system contains both offline-profiling and online-adjustment. During the offline profiling phase, we extract workload characteristics from the model architecture and weights, collect a small number of measurements, and calibrate the analytical latency and energy models. The current implementation requires only 18 samples for latency modeling and 30 samples for energy modeling. We then precompute the energy-minimizing configuration for each sequence length and latency budget. During online inference, the system retrieves the corresponding configuration in constant time for each generated token and jointly adjusts the computing configuration (processor/memory frequency, number of active cores). This design avoids expensive online exploration while preserving token-level adaptation.

Our contributions are summarized as follows:

- We have constructed a large-scale dataset<sup>1</sup> for characterizing the latency and energy behavior of LLMs during the decoding phase. The dataset spans Jetson AGX Orin (CPUs and GPUs), Raspberry Pi and Xiaomi tablet, covering 252 power strategies, seven models (LLaMA-1/3/8B and Qwen-0.5/1.5/3/7B), two quantization types (INT4 and FP16), and input lengths from 1 to 1024 tokens.

<sup>1</sup>Dataset and code will be released upon paper acceptance.

- We develop a joint latency-energy analytical model tailored to the characteristics of LLM inference. It is lightweight, generalizable across hardware types, and requires minimal calibration.
- We conduct extensive experiments across heterogeneous hardware platforms, model configurations, and inference backends. Under a 15% latency-relaxation budget, our method reduces energy consumption by up to 26.78% for LLaMA-3-8B inference on GPU, with only a 6.40% latency increase.

## II. RELATED WORK

**DVFS-based energy optimization.** Dynamic voltage and frequency scaling (DVFS) provides a system-level approach to energy efficiency by dynamically adjusting processor voltage and frequency at runtime. Existing DVFS control methods can be broadly categorized into model-free and model-based strategies. Model-free approaches [10]–[14] typically rely on runtime signals, such as performance counters, processor utilization, and execution time, and often employ reinforcement learning or adaptive control to determine operating configurations. In contrast, model-based approaches [8], [15], [16] rely on offline profiling and predictive latency or energy models, providing lower runtime overhead and more stable control.

Although DVFS has been extensively studied for conventional workloads, including multimedia applications [17], mobile gaming [18], [19], and autonomous systems [20], its application to LLM inference remains limited, particularly on edge devices. Existing DVFS schemes commonly operate at coarse control granularities, ranging from several seconds [11]–[14] to the entire application runtime [10], [21]–[23], making it difficult to capture the sequence-dependent temporal variations of autoregressive LLM decoding. Moreover, learning-based DVFS methods [23]–[27] generally require substantial training data and may exhibit limited generalization across hardware platforms and workload types.

Recent studies have extended DVFS to DNN and LLM inference from different perspectives. DVFO [28] jointly optimizes CPU, GPU, and memory frequencies together with feature-map offloading in an edge–cloud setting, while SparseDVFS [29] exploits operator sparsity and aggregates operators into super-blocks to reduce DVFS transition overhead. Han et al. [30] develop a GPU-frequency-aware latency model by partitioning conventional DNNs into execution blocks. These methods, however, are designed around edge–cloud offloading, operator sparsity, or feed-forward DNN execution graphs, and do not directly capture the workload evolution and KV-cache growth of autoregressive LLM decoding.

Power-efficient LLM inference has also been studied in cloud platforms [1], [2], [31]; throttLL’eM [32] predicts future KV-cache usage and batch size to jointly scale GPU frequency and instance size; GreenLLM [9] separately controls the prefill and decode phases, and DynamoLLM [31] dynamically adjusts serving instances, model parallelism, and GPU frequency at the cluster level. These systems rely on cloud-serving mechanisms such as request batching, workload prediction, instance

scaling, job routing, or multi-GPU deployment, which differ substantially from standalone edge inference. Camel [33] is closer to our setting and formulates GPU-frequency and batch-size selection as a multi-armed bandit problem on a Jetson platform. In contrast to Camel’s online exploration, our method uses lightweight analytical models and offline precomputation, while jointly considering the number of active compute units, compute frequency, and memory frequency. Therefore, Camel and GreenLLM are included as representative experimental baselines in Section IV-D, whereas the remaining methods are compared qualitatively due to their different deployment assumptions and optimization dimensions.

**Latency and energy modeling for LLM inference.** Accurate latency and energy models provide a critical foundation for predictive DVFS control. Zeus [15] focuses on DVFS-aware energy optimization for DNN training in datacenter environments. Imai et al. [16] investigate LLM inference-latency prediction under fixed output-length assumptions, while Wang et al. [8] propose operator-level analytical latency and power models for specialized neural processing units. Although these studies provide valuable insights, they primarily focus on training, fixed workloads, specialized accelerators, or conventional DNN execution, rather than the dynamically evolving compute and memory behavior of autoregressive LLM inference on general-purpose edge hardware.

**Our work.** Our work introduces a joint latency–energy modeling and optimization framework for autoregressive LLM inference on edge devices, targeting general-purpose CPUs and GPUs. Instead of relying on workload prediction, reinforcement-learning policy training, operator sparsity, or cloud resources, our framework models computation, dequantization, memory traffic, compute–memory overlap, and hardware power consumption. It precomputes energy-optimal configurations indexed by sequence length and latency budget, enabling constant-time lookup and low-overhead per-token adjustment across heterogeneous edge platforms and inference backends.

## III. METHODOLOGY

### A. Problem Setup

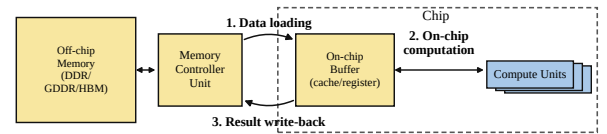


Fig. 1. Abstracted hardware execution flow of LLM inference.

Figure 1 illustrates the abstracted hardware execution flow of LLM inference, including data loading, on-chip computation, and result write-back. This abstraction ensures that the proposed framework remains compatible with various Von Neumann architectures, including CPUs, GPUs, and NPUs. Inference performance is determined by the interplay between memory bandwidth and computational efficiency.

**Roofline Motivation.** To quantify the performance bottleneck of autoregressive decoding, we adopt the Roofline model, where the achievable throughput is bounded by

$$\Pi = \min(\Pi_{\text{peak}}, B_{\text{max}} \text{AI}), \quad \text{AI}_{\text{crit}} = \frac{\Pi_{\text{peak}}}{B_{\text{max}}}. \quad (1)$$

Here,  $\Pi_{\text{peak}}$  is the peak compute throughput,  $B_{\text{max}}$  is the maximum memory bandwidth, and AI denotes arithmetic intensity in operations per byte. For the Jetson AGX Orin,  $\Pi_{\text{peak}} = 10.6 \times 10^{12}$  OP/s and  $B_{\text{max}} = 204.8 \times 10^9$  Bytes/s, yielding  $\text{AI}_{\text{crit}} \approx 52$  OP/Byte. In contrast, the arithmetic intensities of all analyzed operators in LLaMA-3-8B-FP16 decoding range from 0.12 to 3.74 OP/Byte, substantially below this threshold. Therefore, decoding is primarily constrained by memory bandwidth rather than peak compute capability. This memory-bound behavior creates opportunities to reduce the compute frequency, and hence energy consumption, with limited latency degradation.

Since the response latency of DVFS is typically on the order of tens of milliseconds, we propose a per-token frequency adaptation strategy, rather than per-operator control at the microsecond scale or per-query control at the second scale. Meanwhile, due to the highly consistent operator structure during the decoding phase and the relatively stable incremental memory pressure caused by KV-cache growth, the optimal strategy often remains stable over a certain sequence interval. This design enables the framework to capture energy-efficient operating points at token-level granularity, while avoiding the performance penalties caused by overly frequent DVFS transitions.

Formally, we define a power strategy as  $\Phi = (n_{\text{comp}}, f_{\text{comp}}, f_{\text{mem}})$ , where  $n_{\text{comp}}$  denotes the number of active compute units,  $f_{\text{comp}}$  is the compute frequency, and  $f_{\text{mem}}$  represents the memory frequency. For an autoregressive inference task with a target generation length  $L$ , our goal is to determine an optimal strategy sequence  $\{\Phi_1, \Phi_2, \dots, \Phi_L\}$  to maximize energy efficiency under specified latency constraints.

## B. Latency Modeling

Total inference latency  $T_{\text{total}}$  combines compute latency  $T_{\text{comp}}$  and memory access latency  $T_{\text{mem}}$ , which may overlap partially due to hardware prefetching mechanisms that parallelize computation and data fetching.

1) *Computational Latency:* Inference involves activation and, if applicable, weight dequantization. Activation primarily consists of floating-point multiply-accumulate (MAC) operations, while dequantization includes non-arithmetic operations such as reshaping, bit shifting, and type conversion. To improve modeling flexibility, we decouple the two. Let  $\mathcal{N}_{\text{op}}$  be the number of activation operations and  $\mathcal{N}_{\text{dq}}$  be the number of weights to dequantize. The raw compute time is:

$$T_{\text{comp}} = \frac{\mathcal{N}_{\text{op}}}{T_{p_{\text{op}}}} + \frac{\mathcal{N}_{\text{dq}}}{T_{p_{\text{dq}}}}, \quad (2)$$

where  $T_{p_{\text{op}}}$  and  $T_{p_{\text{dq}}}$  are the activation and inverse quantization throughput, respectively. Since throughput is inherently

hardware-dependent and varies significantly across architectures, we introduce two efficiency factors to generalize across platforms:

- **Activation Efficiency**  $C_{\text{ce}}$ : effective activation ops per unit per cycle.
- **Dequantization Efficiency**  $C_{\text{dqe}}$ : effective dequantized weights per unit per cycle.

These efficiency terms abstract away low-level architectural details, allowing us to reflect key hardware traits such as instruction-level parallelism and cache behavior. For instance, on CPUs, these efficiencies may be influenced by the instruction per cycle (IPC) rate and Single Instruction Multiple Data (SIMD) width, while on NVIDIA GPUs they may reflect the number of streaming multiprocessors and specialized tensor cores. Both  $C_{\text{ce}}$  and  $C_{\text{dqe}}$  can be empirically calibrated through microbenchmarking on the target hardware platform.

With  $n_{\text{comp}}$  active units at frequency  $f_{\text{comp}}$ , the computational latency becomes:

$$T_{\text{comp}} = \frac{\mathcal{N}_{\text{op}}}{n_{\text{comp}} f_{\text{comp}} C_{\text{ce}}} + \frac{\mathcal{N}_{\text{dq}}}{n_{\text{comp}} f_{\text{comp}} C_{\text{dqe}}}. \quad (3)$$

2) *Memory Latency:* Memory latency  $T_{\text{mem}}$  captures the time to transfer data (e.g., weights, activations and intermediate states) between memory and compute units. It is modeled as

$$T_{\text{mem}} = \frac{\mathcal{M}_{\text{mem}}}{B_{\text{bandwidth}}}, \quad (4)$$

where  $\mathcal{M}_{\text{mem}}$  is the total data volume and  $B_{\text{bandwidth}}$  is the effective memory bandwidth.

$B_{\text{bandwidth}}$  is determined by several architectural parameters. Specifically, the theoretical peak bandwidth (often called the burst rate) scales with the base clock frequency  $f_{\text{mem}}$ , the number of data transfers per cycle  $\eta$ , the bus width  $\omega$  and the number of channels  $N_{\text{ch}}$ :

$$B_{\text{bandwidth}}^{\text{peak}} \propto f_{\text{mem}} \eta \omega N_{\text{ch}}.$$

However, this peak is seldom sustained in practice due to cache hierarchies, access patterns and controller behavior.

To bridge theory and practice, we introduce a **memory efficiency factor**  $C_{\text{me}}$  that abstracts both hardware features (e.g. burst transfers, channel interleaving, cache effects) and workload-specific access behavior. We then write

$$B_{\text{bandwidth}} = f_{\text{mem}} C_{\text{me}}. \quad (5)$$

This formulation captures the effective data-transfer capability of the memory subsystem under LLM-specific memory access patterns, in contrast to conventional models that rely solely on physical specifications.  $C_{\text{me}}$  can also be calibrated via microbenchmarking on the target hardware.

Substituting Equation 5 into Equation 4 yields

$$T_{\text{mem}} = \frac{\mathcal{M}_{\text{mem}}}{f_{\text{mem}} C_{\text{me}}}. \quad (6)$$

3) *Latency Overlap*: Modern processors employ latency-hiding techniques to mitigate memory bottlenecks via parallel execution of compute and memory operations. During LLM inference, stages such as weight loading and activation propagation often overlap with matrix multiplications and attention computations. To quantify this concurrency, we introduce an overlap model that measures the extent to which memory latency is masked by overlapping compute tasks.

We define a dimensionless *overlap factor*  $\gamma \in [0, 1]$ , representing the fraction of memory latency overlapped with the computation. The effective latency overlap becomes:

$$T_{\text{overlap}} = \gamma T_{\text{mem}}, \quad (7)$$

where  $T_{\text{mem}}$  is the standalone memory latency, and  $T_{\text{overlap}}$  reflects the overlapped part on the critical path.

However, the extent of overlap is not fixed, it is inherently dependent on both hardware microarchitecture and the runtime characteristics of the workload, such as memory traffic volume, instruction-level parallelism, and pipeline depth. The design of  $\gamma$  is guided by the following empirical observations rooted in practical inference workloads:

1. *Increased memory traffic* (larger  $\mathcal{M}_{\text{mem}}$ ) or *slower memory subsystems* (lower  $f_{\text{mem}}$ ) reduce the ability to hide latency, resulting in increasing  $T_{\text{mem}}$  and decreasing  $\gamma$ .

2. *Higher computational load* (e.g., larger  $\mathcal{N}_{\text{op}}$  or  $\mathcal{N}_{\text{dq}}$ ) increases the opportunity for overlap, especially if the computational latency dominates the timeline. *Lower compute frequency* ( $f_{\text{comp}}$ ) tends to stretch the compute phase, enabling better pipelining with memory access and thus increasing  $T_{\text{comp}}$  and  $\gamma$ .

Rather than modeling  $\gamma$  as an explicit function of low-level variables ( $\mathcal{M}_{\text{mem}}$ ,  $f_{\text{mem}}$ ,  $\mathcal{N}_{\text{op}}$ , etc.), we express it as a sigmoid function of the memory-to-compute latency ratio  $T_{\text{mem}}/T_{\text{comp}}$ , which provides an intuitive and hardware-agnostic abstraction:

$$\gamma = \text{Sigmoid}\left(-\gamma_1\left(\frac{T_{\text{mem}}}{T_{\text{comp}}} - \gamma_2\right)\right) = \frac{1}{1 + \exp\left(\gamma_1\left(\frac{T_{\text{mem}}}{T_{\text{comp}}} - \gamma_2\right)\right)}, \quad (8)$$

where  $\gamma_1$  controls the transition sharpness and  $\gamma_2$  denotes the critical threshold separating memory- and compute-bound regimes.

As shown in Figure 2,  $\gamma$  declines when  $T_{\text{mem}}/T_{\text{comp}} \gg \gamma_2$ , indicating memory-bound behavior, and approaches a stable high value ( $\gamma \rightarrow 1/(1 + \exp(-\gamma_1\gamma_2))$ ) when  $T_{\text{mem}}/T_{\text{comp}} \ll \gamma_2$ , denoting dominant compute phases, showing substantial overlap. This abstraction unifies architectural latency-hiding features (e.g., out-of-order execution, speculative loads, non-blocking caches) with workload-level interleaving patterns in LLM inference. It captures both static hardware capabilities and dynamic temporal concurrency, enabling accurate latency estimation across platforms without resorting to platform-specific heuristics.

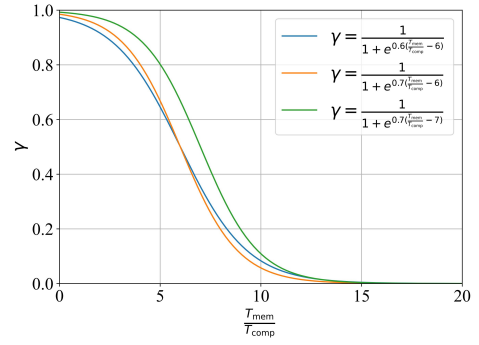


Fig. 2. Curve of the overlap factor  $\gamma$ .

We denote the time consumed by serial operations such as operator scheduling, kernel launching, and data preprocessing as the overhead time  $T_{\text{overhead}}$ . Combining all terms (Equations 3, 6, and 8), we derive the complete latency model:

$$T_{\text{total}} = \frac{\mathcal{N}_{\text{op}}}{n_{\text{comp}} f_{\text{comp}} \underline{C}_{\text{ce}}} + \frac{\mathcal{N}_{\text{dq}}}{n_{\text{comp}} f_{\text{comp}} \underline{C}_{\text{dqe}}} + \left(1 - \frac{1}{1 + \exp\left(\gamma_1\left(\frac{T_{\text{mem}}}{T_{\text{comp}}} - \gamma_2\right)\right)}\right) \frac{\mathcal{M}_{\text{mem}}}{f_{\text{mem}} \underline{C}_{\text{me}}} + \underline{T_{\text{overhead}}}. \quad (9)$$

The underlined parameters are hardware-specific attributes that can be measured via microbenchmarking on the target platform.

### C. Power Consumption Modeling

The processor's total power consumption  $P_{\text{total}}$  comprises both static and dynamic components:

$$P_{\text{total}} = P_{\text{static}} + P_{\text{dynamic}} = P_{\text{static}} + P_{\text{comp}} + P_{\text{mem}}. \quad (10)$$

**Static power**  $P_{\text{static}}$  originates from subthreshold leakage currents in transistors. While influenced by fabrication process, supply voltage, and junction temperature [34]–[36], it remains invariant with respect to workload intensity and operating frequency. In our formulation,  $P_{\text{static}}$  is treated as the maximum leakage observed during inference, corresponding to a worst-case thermal scenario under active power management.

**Dynamic compute power**  $P_{\text{comp}}$  is determined by switching activity in logic gates and scales with both frequency and voltage [35]. The per-unit dynamic power follows:

$$P_{\text{comp}} = A C_L V_{DD}^2 f_{\text{comp}}, \quad (11)$$

where  $A$  denotes the switching activity factor,  $C_L$  the effective load capacitance,  $V_{DD}$  the supply voltage, and  $f_{\text{comp}}$  the working frequency of compute units.

To abstract hardware-specific physical parameters while preserving fidelity, we introduce a **dynamic power coefficient**:

$$\alpha = A C_L V_{DD}^2, \quad (12)$$

yielding a scalable form:

$$P_{\text{comp}} = \alpha f_{\text{comp}} n_{\text{comp}}, \quad (13)$$

where  $n_{\text{comp}}$  denotes the number of active compute units. This formulation enables tractable analysis of DVFS and parallelism effects on power consumption.

**Dynamic memory power**  $P_{\text{mem}}$  exhibits different scaling behavior, dominated by I/O activity, prefetching, and memory controller behavior. We model it as a linear function of memory frequency:

$$P_{\text{mem}} = \beta f_{\text{mem}}, \quad (14)$$

where  $\beta$  captures architectural characteristics such as bus width, I/O driver capacitance, and burst prefetch mechanisms.

In addition, modern processors often include multiple compute units ( $N_{\text{comp}}$ ), of which only a subset are active during inference. Idle units typically enter reduced-power states but still contribute to leakage. We model this residual idle power as:

$$P_{\text{idle-total}} = (N_{\text{comp}} - n_{\text{comp}}) P_{\text{comp-idle}}, \quad (15)$$

with  $P_{\text{comp-idle}}$  denoting the per-unit idle power.

Combining all terms, we arrive at the complete power consumption model:

$$P_{\text{total}} = \underline{P}_{\text{static}} + \underline{\alpha} f_{\text{comp}} n_{\text{comp}} + (N_{\text{comp}} - n_{\text{comp}}) \underline{P}_{\text{comp-idle}} + \underline{\beta} f_{\text{mem}}. \quad (16)$$

Similarly, the underlined parameters can be measured via microbenchmarking on the target platform. Equation (9) and (16) strike a balance between physical realism and analytical simplicity, offering a theoretically grounded framework for optimizing energy-performance trade-offs.

#### D. End-to-End Energy Optimization

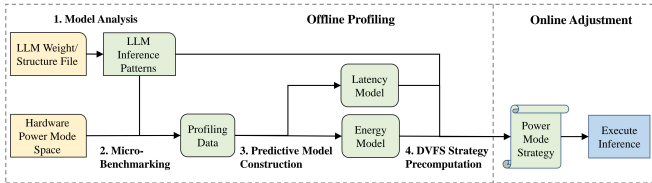


Fig. 3. Overview of the energy optimization workflow.

To enable energy-efficient deployment of LLMs on power-constrained edge devices, we propose an end-to-end optimization framework that jointly minimizes inference energy while respecting latency constraints. As shown in Figure 3, the workflow comprises two complementary phases: an *offline profiling* phase for model profiling, hardware calibration and policy generation, and an *online adjustment* phase for runtime control guided by precomputed decisions.

**Offline Profiling.** This phase constructs parameterized latency and energy models specific to the target hardware. It consists of four key steps:

*Step 1: Model Analysis.* We extract workload-specific quantities from the LLM architecture and weight files, including: (i) the number of activation operations  $\mathcal{N}_{\text{op}}$ , (ii) the number of weights to dequantize  $\mathcal{N}_{\text{dq}}$ , and (iii) the total memory traffic

volume  $\mathcal{M}_{\text{mem}}$  for each decoding step. These quantities are computed based on static model metadata (e.g., size of hidden states, number of layers, quantization scheme), enabling a structural understanding of the workload without requiring actual inference execution.

*Step 2: Micro-Benchmarking.* To characterize hardware efficiency under diverse strategies, we define the empirical search space:

$$\mathcal{H} = \mathcal{L} \times \mathcal{N}_{\text{comp}} \times \mathcal{F}_{\text{comp}} \times \mathcal{F}_{\text{mem}}, \quad (17)$$

where  $\mathcal{L}$  denotes supported sequence lengths, and  $\mathcal{N}_{\text{comp}}$ ,  $\mathcal{F}_{\text{comp}}$ ,  $\mathcal{F}_{\text{mem}}$  represent candidate compute unit counts and DVFS settings. For robust model fitting, we adopt a rule-of-thumb: the number of samples should exceed the number of model parameters by at least a factor of 3. Specifically, we collect 18 data points for latency modeling (6 parameters) and 30 for energy modeling (6 parameters for latency model and 4 parameters for power model), uniformly sampled from  $\mathcal{H}$ .

*Step 3: Predictive Model Calibration.* Nonlinear least-squares regression is employed to fit the analytical latency and energy models present above. Here, the total inference latency adheres to Equation (9), and the total energy consumption is determined by  $E_{\text{total}} = P_{\text{total}} T_{\text{total}}$ , with  $P_{\text{total}}$  following Equation (16). This approach yields a lightweight yet expressive performance model, designed to support efficient design-space exploration.

*Step 4: DVFS-Aware Strategy Precomputation.* Leveraging the calibrated models, we perform exhaustive evaluation across the strategy space  $\mathcal{H}$  to identify optimal DVFS strategies for each decoding step. For a given sequence length  $L_t$ , we compute the latency  $T_{\text{total}}(\Phi)$  and energy  $E_{\text{total}}(\Phi)$  under all feasible strategies  $\Phi$ , and solve:

$$\begin{aligned} \Phi^*(L_t, r\%) &= \arg \min_{\Phi} E_{\text{total}}(\Phi) \\ \text{s.t. } T_{\text{total}}(\Phi) &\leq (1 + r\%) T_{\text{total}}(\Phi_0), \end{aligned} \quad (18)$$

where  $\Phi_0$  denotes the minimal-latency strategy  $(n_{\text{comp}}^{\text{max}}, f_{\text{comp}}^{\text{max}}, f_{\text{mem}}^{\text{max}})$ , and  $r\%$  specifies the allowable latency relaxation. The resulting optimal strategies  $\{\Phi^*(L_t, r\%)\}$  are stored in a lookup table, enabling constant-time retrieval during inference.

*a) Online Adjustment.* At runtime, energy-aware control is enacted through three lightweight steps: context setup, strategy lookup, and dynamic reconfiguration. First, the pre-computed policy table  $\mathcal{C} : (L_t, r\%) \mapsto \Phi^*(L_t, r\%)$  is loaded into memory. At each decoding step, the current sequence length  $L_t$  and latency budget  $r\%$  are used to index the optimal strategy  $\Phi^*$ , which can be retrieved in constant time. Then, the system applies the selected strategy by adjusting processor and memory frequencies via standard DVFS interfaces (e.g., Linux ‘cpufreq’), and scaling the number of active compute units  $n_{\text{comp}}$  within the inference framework.

This end-to-end design enables low-overhead, per-token power strategy adaptation, dynamically optimizing energy consumption while adhering to strict latency constraints. By

TABLE I  
MODELING PERFORMANCE OF LATENCY, POWER, AND ENERGY FOR LLAMA MODELS ON JETSON.

| Device         | Model             | Latency       |              | Power         |              | Energy        |              |
|----------------|-------------------|---------------|--------------|---------------|--------------|---------------|--------------|
|                |                   | $R^2$         | MAPE         | $R^2$         | MAPE         | $R^2$         | MAPE         |
| CPU            | Llama-3.2-1B-Q4_0 | 0.9791        | 4.02%        | 0.9470        | 5.03%        | 0.9656        | 4.92%        |
|                | Llama-3.2-1B-FP16 | 0.9843        | 3.50%        | 0.9305        | 5.33%        | 0.9806        | 4.66%        |
|                | Llama-3.2-3B-Q4_0 | 0.9599        | 4.31%        | 0.9613        | 5.02%        | 0.9659        | 4.91%        |
|                | Llama-3.2-3B-FP16 | 0.9792        | 3.26%        | 0.9562        | 5.19%        | 0.9791        | 4.89%        |
|                | Llama-3-8B-Q4_0   | 0.9705        | 4.09%        | 0.9422        | 5.00%        | 0.9688        | 4.35%        |
|                | Llama-3-8B-FP16   | 0.9722        | 4.00%        | 0.9140        | 6.40%        | 0.9676        | 5.38%        |
|                | Llama-2-13B-Q4_0  | 0.9678        | 4.28%        | 0.9356        | 5.41%        | 0.9612        | 4.78%        |
|                | Llama-2-13B-FP16  | 0.9765        | 3.75%        | 0.9289        | 5.82%        | 0.9723        | 4.67%        |
| <b>Average</b> |                   | <b>0.9737</b> | <b>3.90%</b> | <b>0.9395</b> | <b>5.40%</b> | <b>0.9701</b> | <b>4.82%</b> |
| GPU            | Llama-3.2-1B-Q4_0 | 0.9958        | 5.71%        | 0.9335        | 5.47%        | 0.9889        | 4.87%        |
|                | Llama-3.2-1B-FP16 | 0.9993        | 3.95%        | 0.9504        | 4.37%        | 0.9973        | 2.94%        |
|                | Llama-3.2-3B-Q4_0 | 0.9962        | 6.45%        | 0.9176        | 6.25%        | 0.9938        | 4.24%        |
|                | Llama-3.2-3B-FP16 | 0.9994        | 2.48%        | 0.9738        | 4.33%        | 0.9943        | 3.21%        |
|                | Llama-3-8B-Q4_0   | 0.9997        | 2.22%        | 0.9871        | 3.37%        | 0.9960        | 3.35%        |
|                | Llama-3-8B-FP16   | 1.0000        | 0.85%        | 0.9942        | 2.47%        | 0.9972        | 2.45%        |
|                | Llama-2-13B-Q4_0  | 0.9975        | 3.45%        | 0.9587        | 4.62%        | 0.9921        | 3.55%        |
|                | Llama-2-13B-FP16  | 0.9998        | 1.28%        | 0.9856        | 2.95%        | 0.9965        | 2.67%        |
| <b>Average</b> |                   | <b>0.9985</b> | <b>3.30%</b> | <b>0.9626</b> | <b>4.23%</b> | <b>0.9945</b> | <b>3.41%</b> |

tailoring power settings to each generated token, the system balances control overhead with real-time responsiveness. Moreover, since the incremental memory pressure introduced by KV-cache growth is relatively stable, the optimal configuration usually remain stable for a long sequence length and thus the system can avoid the performance penalties caused by overly frequent DVFS transitions.

#### IV. EXPERIMENTS

We conduct extensive empirical studies in two parts: (1) **modeling accuracy evaluation** using a large-scale, system-atically collected dataset, and (2) **end-to-end optimization assessment** under realistic deployment conditions.

##### A. Evaluation Platforms, Baselines & Metrics

The experiments span three representative edge platforms: Jetson AGX Orin, Raspberry Pi 4B, and Xiaomi Pad 6 Pro, representing a high-performance edge platform, a resource-constrained platform, and a commercial mobile device, respectively. We investigate the effects of hardware configurations, model architectures, quantization schemes, and context lengths on modeling accuracy and optimization effectiveness. We use two inference backends, the open-source engine `llama.cpp` [37] and TensorRT-LLM, to evaluate the generalization of the proposed framework across different LLM inference software stacks.

For the controlled comparison on Jetson AGX Orin, we compare our method with representative DVFS strategies, including GreenLLM [9], Camel-Freq [33], and the system-default CPU and GPU governors. For GreenLLM, we reproduce its online frequency-scaling policy during the decoding phase. For Camel, since the batch size is fixed to one in all experiments, its original batch-size optimization is disabled,

and only its frequency-selection action is retained; we refer to this adapted baseline as Camel-Freq. The system-default CPU and GPU governors are `schedutil` and `nvhost_podgov`, respectively. All practical strategies are evaluated using the same models, workloads, hardware platform, and supported operating ranges. We additionally report an Oracle that exhaustively selects the best measured configuration from the same candidate space, providing an empirical upper bound.

We define the maximum-performance reference configuration as  $\Phi_0 = (n_{\text{comp}}^{\text{max}}, f_{\text{comp}}^{\text{max}}, f_{\text{mem}}^{\text{max}})$ , where  $n_{\text{comp}}^{\text{max}}$  denotes the maximum number of active compute units, and  $f_{\text{comp}}^{\text{max}}$  and  $f_{\text{mem}}^{\text{max}}$  denote the maximum compute and memory frequencies, respectively. This configuration provides the minimum measured latency within the supported operating space. All reported latency increases, energy reductions, and EDP improvements are measured relative to  $\Phi_0$ .

To jointly evaluate inference energy and latency, we use the normalized energy–delay product (EDP):

$$\text{EDP}_{\text{norm}} = (1 - \Delta E)(1 + \Delta T), \quad (19)$$

where  $\Delta E$  and  $\Delta T$  denote the energy reduction and latency increase relative to  $\Phi_0$ , respectively. A lower  $\text{EDP}_{\text{norm}}$  indicates a better energy–performance trade-off, and a value below 1 represents an improvement over  $\Phi_0$ . For visualization, we further report the EDP improvement:

$$\text{EDP}_{\text{imp}} = 100(1 - \text{EDP}_{\text{norm}})\%, \quad (20)$$

where a larger value indicates a better energy–performance trade-off.

##### B. Dataset Construction

We construct a large-scale dataset on Jetson AGX Orin, capturing latency, power, and energy metrics under diverse

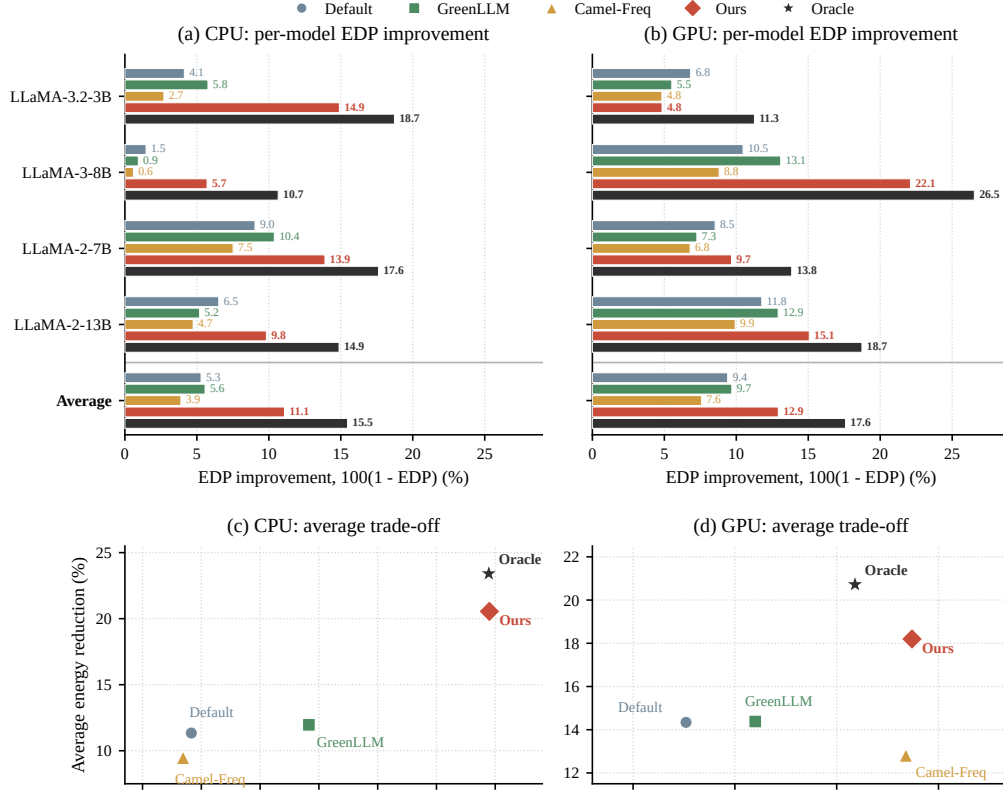


Fig. 4. Comparison of DVFS strategies on Jetson AGX Orin under the constraint of 15% latency increase.

runtime settings: leftmargin=1.5em

- **Compute units**  $\mathcal{N}_{\text{comp}}$ : for CPU inference,  $\mathcal{N}_{\text{comp}} \in \{4, 8, 12\}$ ; for GPU,  $\mathcal{N}_{\text{comp}} = 1$ .
- **Processor frequency**  $\mathcal{F}_{\text{comp}}$ : 17 high CPU frequencies from the original 29 ones (excluding small ones due to limited energy-latency efficiency) and all 11 GPU frequencies.
- **Memory frequency**  $\mathcal{F}_{\text{mem}}$ : All 4 supported memory frequencies.
- **Model architectures and sizes**: LLaMA [38], [39] and Qwen [40] models from 0.5B to 13B.
- **Weight quantization**: Half-precision floating-point and 4-bit weight-only quantized models.
- **Context lengths**  $\mathcal{L}$ : 1 to 1024 tokens.

Each configuration is evaluated across multiple context lengths and power strategies  $\Phi$ , averaged over 10 repetitions to reduce noise. Power is measured from platform-specific sysfs interfaces: INA3221 power-rail readings on Jetson AGX Orin, with CPU/GPU frequencies logged via cpufreq and KGSL/devfreq. The dataset comprises 3,612,672 samples collected over five months.

### C. Modeling Accuracy Evaluation

We assess modeling accuracy using two standard regression metrics: the coefficient of determination ( $R^2$ ) and mean absolute percentage error (MAPE). As detailed in Section III-D, each model uses 18 profiling samples to calibrate the latency parameters and 30 samples to calibrate the complete

TABLE II  
ABLATION OF OVERLAP FUNCTIONS FOR LATENCY PREDICTION.

| Overlap Function        | ERR < 2%      | ERR < 5%      | ERR < 10%     |
|-------------------------|---------------|---------------|---------------|
| <b>Sigmoid (ours)</b>   | <b>33.23%</b> | <b>63.49%</b> | <b>91.21%</b> |
| Linear                  | 11.48%        | 30.35%        | 61.67%        |
| Constant $\gamma = 0$   | 6.18%         | 15.24%        | 36.49%        |
| Constant $\gamma = 0.5$ | 18.46%        | 42.79%        | 73.52%        |
| Constant $\gamma = 1$   | 1.98%         | 4.89%         | 14.45%        |

latency–energy model. The samples are uniformly drawn from the configuration space in Equation (17), and the model parameters are fitted using nonlinear least squares.

Table I reports the modeling results for LLaMA models on Jetson. On CPU, the average  $R^2$  scores for latency, power, and energy are 0.9737, 0.9395, and 0.9701, respectively; on GPU, they reach 0.9985, 0.9626, and 0.9945. Most energy predictions for the evaluated LLaMA workloads have errors below 10%. Qwen models exhibit comparable accuracy, with average energy MAPEs of 5.40% on CPU and 4.23% on GPU, demonstrating generalization across model families.

**Evaluation of the overlap function.** We further evaluate the overlap function used in the latency model by comparing the proposed sigmoid formulation with a linear function and three constant settings,  $\gamma \in \{0, 0.5, 1\}$ . Here,  $\text{ERR} < x\%$  denotes the percentage of samples whose latency-prediction error is below  $x\%$ .

As shown in Table II, our sigmoid formulation consistently achieves the highest accuracy, with 63.49% and 91.21% of predictions falling within 5% and 10% error, respectively. For

91% of the cases, the latency prediction error is less than 10%. Compared with other methods, our sigmoid overlap function substantially improves latency-prediction accuracy over both the linear formulation and all constant settings.

#### D. Comparison with Existing DVFS Strategies

We evaluate our method against representative DVFS strategies on the Jetson AGX Orin platform. The experiments use FP16 LLaMA models with `llama.cpp` and a batch size of one, consistent with the single-request autoregressive decoding scenario targeted by our framework. For each workload, we generate 1,024 tokens and report the average results over ten runs. Using the maximum-performance reference configuration  $\Phi_0$  defined in the experimental setup, we set the latency-relaxation budget to  $r = 15\%$ . All reported latency increases, energy reductions, and EDP improvements are measured relative to  $\Phi_0$ .

As shown in Fig. 4(a)–(b), our method achieves the highest average EDP improvement among all practical strategies on both CPU and GPU. On CPU, our method achieves an average EDP improvement of 11.1%, compared with 5.3%, 5.6%, and 3.9% for the default governor, GreenLLM, and Camel-Freq, respectively. On GPU, our method achieves an average EDP improvement of 12.9%, compared with 9.4%, 9.7%, and 7.6% for the three baselines. The per-model results further show that our method achieves the best practical EDP for all four CPU workloads and three of the four GPU workloads.

Fig. 4(c)–(d) further illustrates the average latency–energy trade-off. On CPU, our method reduces energy consumption by 20.55%, substantially exceeding the 11.34%, 11.96%, and 9.47% reductions achieved by the default governor, GreenLLM, and Camel-Freq, respectively. Although this reduction is accompanied by an average latency increase of 11.90%, the greater energy savings yield the best overall EDP among the practical strategies. On GPU, our method reduces energy consumption by 18.20% with an average latency increase of 6.87%. This latency overhead is nearly identical to Camel-Freq’s 6.84%, while our method provides an additional 5.39% of energy savings. Consequently, the normalized EDP decreases from 0.9241 (for Camel-Freq) to 0.8708 (our method), confirming that the improvement does not merely result from allowing a larger latency overhead. These aggregate and per-model results consistently demonstrate a better energy–performance trade-off across the evaluated workloads.

The Oracle achieves average EDP improvements of 15.5% on CPU and 17.6% on GPU. Our method remains within approximately 5.2% and 5.7% of the Oracle EDP on CPU and GPU, respectively, while requiring only a small number of profiling samples and constant-time online policy lookup. These results demonstrate that the proposed lightweight model-guided strategy approaches exhaustive search without incurring its prohibitive profiling and runtime overhead.

For completeness, Table III reports the detailed latency and energy results of our method for each evaluated FP16 LLaMA workload. The largest gain is observed for LLaMA-3-8B on GPU, where our method reduces energy consumption

TABLE III  
LATENCY AND ENERGY TRADE-OFFS FOR FP16 LLaMA MODELS ON JETSON AGX ORIN UNDER A 15% LATENCY-RELAXATION BUDGET.

| Model          | CPU              |                  | GPU              |                  |
|----------------|------------------|------------------|------------------|------------------|
|                | Latency Increase | Energy Reduction | Latency Increase | Energy Reduction |
| LLaMA-3.2-3B   | 12.08%           | 24.07%           | 12.34%           | 13.55%           |
| LLaMA-3-8B     | 13.04%           | 16.58%           | 6.40%            | 26.78%           |
| LLaMA-2-7B     | 10.82%           | 22.31%           | 4.11%            | 13.66%           |
| LLaMA-2-13B    | 11.64%           | 19.23%           | 4.63%            | 18.82%           |
| <b>Average</b> | <b>11.90%</b>    | <b>20.55%</b>    | <b>6.87%</b>     | <b>18.20%</b>    |

by 26.78% with only a 6.40% latency increase, yielding a normalized EDP of 0.7790 and an EDP improvement of 22.1%. The detailed results suggest that more compute- and memory-intensive workloads, which are often associated with larger models, generally expose greater energy-saving opportunities. GPU execution exhibits a more favorable overall energy–performance trade-off: although its average energy reduction is slightly lower than that of CPU, its average latency increase is substantially smaller, leading to a higher average EDP improvement. This observation is consistent with the memory-bound characteristics of autoregressive decoding, which allow GPU frequency scaling to reduce energy consumption with limited performance degradation.

#### E. Generalization across Backends & Devices

To evaluate the generalization of the proposed framework, we conduct cross-backend and cross-device experiments under a unified evaluation protocol. For each workload, we generate 1,024 tokens and report the average results over ten runs. All results are also measured relative to the maximum-performance reference configuration  $\Phi_0$ .

*a) Cross-Backend Generalization.*: To evaluate whether the framework depends on a specific inference engine, we compare its performance under `llama.cpp` and TensorRT-LLM on the same Jetson AGX Orin platform. As shown in Table IV, with TensorRT-LLM, averaged across the evaluated FP16 models and latency-relaxation budgets, our method reduces energy consumption by 14.70% on CPU and 16.64% on GPU, with average latency increases of 9.34% and 6.79%, respectively. Together with the `llama.cpp` results (Table III), these results demonstrate that the proposed latency–energy model and the resulting DVFS policy remain effective across different LLM inference software stacks.

*b) Cross-Device Generalization.*: To evaluate portability across heterogeneous edge hardware, we further deploy the framework with `llama.cpp` on Raspberry Pi 4B and Xiaomi Pad 6 Pro. On Xiaomi Pad 6 Pro, power consumption is derived from battery current and voltage readings, while GPU frequency is logged via KGSL/devfreq. As shown in Table V, our method consistently reduces energy consumption across the evaluated devices, models, quantization categories, and latency-relaxation budgets. Averaged across the evaluated workloads and latency budgets, the method achieves an energy reduction of 6.70% on the Raspberry Pi CPU, with

TABLE IV  
ENERGY OPTIMIZATION RESULTS FOR LLAMA MODELS ON JETSON (TENSORRT-LLM)

| Model             | r%  | CPU              |                  |                                       | GPU              |                  |                                       |
|-------------------|-----|------------------|------------------|---------------------------------------|------------------|------------------|---------------------------------------|
|                   |     | Latency Increase | Energy Reduction | Power Reduction                       | Latency Increase | Energy Reduction | Power Reduction                       |
| Llama-3.2-3B-FP16 | 5%  | 5.12%            | 2.70%            | (10.51 $\pm$ 0.27)%                   | 2.97%            | 7.18%            | (12.96 $\pm$ 0.28)%                   |
|                   | 10% | 8.81%            | 16.36%           | (23.13 $\pm$ 0.23)%                   | 4.98%            | 15.03%           | (19.08 $\pm$ 0.26)%                   |
|                   | 15% | 12.95%           | 25.51%           | (28.05 $\pm$ 0.25)%                   | 10.83%           | 16.99%           | (21.30 $\pm$ 0.29)%                   |
| Llama-3-8B-FP16   | 5%  | 3.57%            | 6.55%            | (9.78 $\pm$ 0.32)%                    | 4.12%            | 12.21%           | (18.53 $\pm$ 0.30)%                   |
|                   | 10% | 7.48%            | 15.25%           | (21.15 $\pm$ 0.26)%                   | 7.63%            | 23.21%           | (27.97 $\pm$ 0.33)%                   |
|                   | 15% | 18.12%           | 21.84%           | (29.83 $\pm$ 0.23)%                   | 10.20%           | 25.21%           | (32.80 $\pm$ 0.25)%                   |
| <b>Average</b>    |     | <b>9.34%</b>     | <b>14.70%</b>    | <b>(20.41 <math>\pm</math> 0.26)%</b> | <b>6.79%</b>     | <b>16.64%</b>    | <b>(22.11 <math>\pm</math> 0.29)%</b> |

TABLE V  
ENERGY OPTIMIZATION ACROSS DEVICES.

| Device           | Model             | Setting | Latency Increase | Energy Reduction | Power Reduction |
|------------------|-------------------|---------|------------------|------------------|-----------------|
| Raspberry Pi CPU | Llama-3.2-1B-Q4_0 | 5%      | 5.18%            | 4.34%            | 8.90%           |
|                  |                   | 10%     | 8.61%            | 5.85%            | 13.18%          |
|                  |                   | 15%     | 9.50%            | 6.23%            | 14.29%          |
|                  | Llama-3.2-1B-FP16 | 5%      | 4.42%            | 9.79%            | 13.56%          |
|                  |                   | 10%     | 7.14%            | 9.52%            | 15.50%          |
|                  |                   | 15%     | 13.30%           | 12.58%           | 19.62%          |
|                  | Llama-3.2-3B-Q4_0 | 5%      | 2.71%            | 2.61%            | 5.08%           |
|                  |                   | 10%     | 6.01%            | 4.41%            | 9.73%           |
|                  |                   | 15%     | 13.69%           | 4.97%            | 16.27%          |
|                  | <b>Average</b>    |         | <b>7.84%</b>     | <b>6.70%</b>     | <b>12.90%</b>   |
| Xiaomi Pad GPU   | Llama-3.2-1B-Q4_0 | 5%      | 4.72%            | 6.04%            | 12.77%          |
|                  |                   | 10%     | 7.41%            | 12.18%           | 17.73%          |
|                  |                   | 15%     | 10.98%           | 13.95%           | 19.84%          |
|                  | Llama-3.2-1B-FP16 | 5%      | 6.82%            | 6.52%            | 12.18%          |
|                  |                   | 10%     | 10.82%           | 14.03%           | 18.02%          |
|                  |                   | 15%     | 15.19%           | 17.37%           | 24.70%          |
|                  | Llama-3.2-3B-Q4_0 | 5%      | 3.28%            | 4.71%            | 6.73%           |
|                  |                   | 10%     | 6.95%            | 9.54%            | 15.98%          |
|                  |                   | 15%     | 10.23%           | 12.61%           | 16.04%          |
|                  | <b>Average</b>    |         | <b>8.49%</b>     | <b>10.77%</b>    | <b>15.01%</b>   |

an average latency increase of 7.84%. On the Xiaomi Pad GPU, it achieves an average energy reduction of 10.77%, with a corresponding latency increase of 8.49%. Although the achievable savings vary with hardware capabilities and workload characteristics, the framework remains effective after lightweight platform-specific calibration, demonstrating its portability across heterogeneous edge devices.

## V. DISCUSSION

In this section, we discuss several implementation issues. The current framework relies on offline-calibrated latency-energy models and precomputed DVFS policies to enable low-overhead runtime adaptation, but it also leaves room for further extension. In particular, the present design selects power strategies at the token level and enforces a local latency constraint for each token, whereas user-perceived latency is typically determined by the end-to-end generation time. This suggests that future work could reformulate the optimization from a global latency-budget perspective, allowing some to-

kens to locally exceed their latency budgets as long as the overall generation latency remains within the user-defined constraint, thereby better exploiting latency slack across tokens for improved energy efficiency.

Meanwhile, our experiments show that the incremental memory pressure introduced by KV-cache growth is relatively stable across consecutive decoding steps, and the optimal DVFS strategy often remains unchanged over a sequence interval. This observation motivates a chunk-level adaptation mechanism that groups consecutive tokens and adjusts DVFS settings at the chunk granularity, reducing frequency-switching overhead and control cost while preserving most of the benefits of fine-grained adaptation.

In addition, the current model assumes relatively stable hardware behavior during inference, while real deployments may experience thermal variation, background activities, and context switching, which can alter effective performance and power characteristics and degrade precomputed policies. Incorporating thermal states and runtime system signals (e.g., CPU cycles, GPU utilization) into the latency-energy model, together with lightweight online adaptation, is therefore a promising direction for improving robustness and generalization under dynamic conditions, multi-stream inference, and heterogeneous hardware scenarios.

## VI. CONCLUSION

This paper presents a lightweight, end-to-end energy optimization framework for LLM inference on edge devices. By modeling latency and energy through analytical modeling and micro-benchmarking, we enable accurate prediction and efficient DVFS-based control under user-defined latency constraints. Extensive experiments on Jetson AGX Orin, Raspberry Pi 4B, and Xiaomi Pad demonstrate the accuracy and generality of our models, as well as the effectiveness of the optimization strategy in reducing energy consumption without significant performance degradation. We further extend our evaluation to both llama.cpp and TensorRT-LLM inference frameworks. Together with experiments across multiple edge devices, these results demonstrate the versatility and applicability of our approach across diverse hardware platforms and inference software stacks.

## REFERENCES

- [1] P. Patel, E. Choukse, C. Zhang, Í. Goiri, B. Warriar, N. Mahalingam, and R. Bianchini, "Characterizing power management opportunities for LLMs in the cloud," in *Proc. ACM ASPLOS*, 2024, pp. 207–222.
- [2] J. Stojkovic, C. Zhang, Í. Goiri, E. Choukse, H. Qiu, R. Fonseca, J. Torrellas, and R. Bianchini, "Tapas: Thermal-and power-aware scheduling for llm inference in cloud platforms," in *Proc. ACM ASPLOS*, 2025, pp. 1266–1281.
- [3] X. Ma, G. Fang, and X. Wang, "Llm-pruner: On the structural pruning of large language models," *Advances in neural information processing systems*, vol. 36, pp. 21 702–21 720, 2023.
- [4] P. Xu, W. Shao, M. Chen, S. Tang, K. Zhang, P. Gao, F. An, Y. Qiao, and P. Luo, "Besas: Pruning large language models with blockwise parameter-efficient sparsity allocation," in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: <https://openreview.net/forum?id=gC6JTEU3jl>
- [5] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale," *Advances in Neural Information Processing Systems*, vol. 35, pp. 30 318–30 332, 2022.
- [6] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning*. PMLR, 2023, pp. 38 087–38 099.
- [7] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for on-device llm compression and acceleration," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 87–100, 2024.
- [8] Z. Wang, Y. Zhang, F. Wei, B. Wang, Y. Liu, Z. Hu, J. Zhang, X. Xu, J. He, X. Wang *et al.*, "Using analytical performance/power model and fine-grained DVFS to enhance AI accelerator energy efficiency," in *Proc. ACM ASPLOS*, 2025, pp. 1118–1132.
- [9] Q. Liu, D. Huang, M. Zapater, and D. Atienza, "Greenllm: Slo-aware dynamic frequency scaling for energy-efficient llm serving," *arXiv preprint arXiv:2508.16449*, 2025.
- [10] J. Guerreiro, A. Ilic, N. Roma, and P. Tomás, "Dvfs-aware application classification to improve gpgpus energy efficiency," *Parallel Computing*, vol. 83, pp. 93–117, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167819118300243>
- [11] P. Zou, A. Li, K. Barker, and R. Ge, "Indicator-directed dynamic power management for iterative workloads on gpu-accelerated systems," in *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*, 2020, pp. 559–568.
- [12] F. Wang, W. Zhang, S. Lai, M. Hao, and Z. Wang, "Dynamic gpu energy optimization for machine learning training workloads," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 11, pp. 2943–2954, 2022.
- [13] F. Wang, M. Hao, W. Zhang, and Z. Wang, "Model-free gpu online energy optimization," *IEEE Transactions on Sustainable Computing*, vol. 9, no. 2, pp. 141–154, 2024.
- [14] Y. Zhang, Q. Wang, Z. Lin, P. Xu, and B. Wang, "Improving gpu energy efficiency through an application-transparent frequency scaling policy with performance assurance," in *Proc. ACM EuroSys*, 2024, pp. 769–785.
- [15] J. You, J.-W. Chung, and M. Chowdhury, "Zeus: Understanding and optimizing GPU energy consumption of DNN training," in *Proc. USENIX NSDI*, 2023, pp. 119–139.
- [16] S. Imai, R. Nakazawa, M. Amaral, S. Choochotkaew, and T. Chiba, "Predicting llm inference latency: A roofline-driven ml method," in *Annual Conference on Neural Information Processing Systems*, 2024.
- [17] Y. Choi, S. Park, and H. Cha, "Graphics-aware power governing for mobile devices," in *Proceedings of the 17th annual international conference on mobile systems, applications, and services*, 2019, pp. 469–481.
- [18] X. Li and G. Li, "An adaptive cpu-gpu governing framework for mobile games on big, little architectures," *IEEE Transactions on Computers*, vol. 70, no. 9, pp. 1472–1483, 2020.
- [19] Y. Choi, S. Park, S. Jeon, R. Ha, and H. Cha, "Optimizing energy consumption of mobile games," *IEEE Transactions on Mobile Computing*, vol. 21, no. 10, pp. 3744–3756, 2021.
- [20] S. Bateni and C. Liu, "NeuOS: A Latency-Predictable Multi-Dimensional optimization framework for DNN-driven autonomous systems," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, Jul. 2020, pp. 371–385. [Online]. Available: <https://www.usenix.org/conference/atc20/presentation/bateni>
- [21] S. Ilager, R. Muralidhar, K. Rammohanrao, and R. Buyya, "A data-driven frequency scaling approach for deadline-aware energy efficient scheduling on graphics processing units (gpus)," in *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*, 2020, pp. 579–588.
- [22] G. Ali, M. Side, S. Bhalachandra, N. J. Wright, and Y. Chen, "An automated and portable method for selecting an optimal gpu frequency," *Future Generation Computer Systems*, vol. 149, pp. 71–88, 2023.
- [23] —, "Performance-aware energy-efficient gpu frequency selection using dnn-based models," in *Proceedings of the 52nd International Conference on Parallel Processing*, 2023, pp. 433–442.
- [24] G. Wu, J. L. Greathouse, A. Lyashevsky, N. Jayasena, and D. Chiou, "Gpgpu performance and power estimation using machine learning," in *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*, 2015, pp. 564–576.
- [25] K. Fan, B. Cosenza, and B. Juurlink, "Predictable gpus frequency scaling for energy and performance," in *Proceedings of the 48th International Conference on Parallel Processing*, ser. ICPP '19. New York, NY, USA: Association for Computing Machinery, 2019. [Online]. Available: <https://doi.org/10.1145/3337821.3337833>
- [26] Q. Wang, C. Liu, and X. Chu, "Gpgpu performance estimation for frequency scaling using cross-benchmarking," in *Proceedings of the 13th Annual Workshop on General Purpose Processing Using Graphics Processing Unit*, ser. GPGPU '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 31–40. [Online]. Available: <https://doi.org/10.1145/3366428.3380767>
- [27] C. Tian, X. Qin, K. Tam, L. Li, Z. Wang, Y. Zhao, M. Zhang, and C. Xu, "CLONE: Customizing LLMs for efficient latency-aware inference at the edge," in *Proceedings of the 2025 USENIX Annual Technical Conference*, ser. USENIX ATC '25. Boston, MA, USA: USENIX Association, 2025, pp. 563–585.
- [28] Z. Zhang, Y. Zhao, H. Li, C. Lin, and J. Liu, "DVFO: Learning-based DVFS for energy-efficient edge-cloud collaborative inference," *IEEE Transactions on Mobile Computing*, vol. 23, no. 10, pp. 9042–9059, 2024.
- [29] Z. Zhang, Z. Wu, J. Liu, and L. Mottola, "SparseDVFS: Sparse-aware DVFS for energy-efficient edge inference," *arXiv preprint arXiv:2603.21908*, 2026.
- [30] Y. Han, Z. Nan, S. Zhou, and Z. Niu, "DVFS-aware DNN inference on GPUs: Latency modeling and performance analysis," in *ICC 2025 – IEEE International Conference on Communications*, 2025, pp. 1274–1279.
- [31] J. Stojkovic, C. Zhang, Í. Goiri, J. Torrellas, and E. Choukse, "Dynamollm: Designing llm inference clusters for performance and energy efficiency," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1348–1362.
- [32] A. K. Kakolyris, D. Masouros, P. Varvaroutsos, S. Xydis, and D. Soudris, "throttLL'eM: Predictive GPU throttling for energy-efficient LLM inference serving," in *Proc. IEEE HPCA*, 2025, pp. 1363–1378.
- [33] H. Xu, L. Peng, S. Song, X. Liu, Ma Jun, S. Li, J. Yu, and X. Mao, "Camel: Energy-aware LLM inference on resource-constrained devices," *arXiv preprint arXiv:2508.09173*, 2025.
- [34] J. A. Butts and G. S. Sohi, "A static power model for architects," in *Proceedings of the 33rd annual ACM/IEEE international symposium on Microarchitecture*, 2000, pp. 191–201.
- [35] H. Sultan, G. Ananthanarayanan, and S. R. Sarangi, "Processor power estimation techniques: a survey," *International Journal of High Performance Systems Architecture*, vol. 5, no. 2, pp. 93–114, 2014.
- [36] G. Singla, G. Kaur, A. K. Unver, and U. Y. Ogras, "Predictive dynamic thermal and power management for heterogeneous mobile platforms," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, 2015, pp. 960–965.
- [37] ggml-org, "llama.cpp: LLM inference in C/C++," 2023, accessed: 2024-09-23. [Online]. Available: <https://github.com/ggml-org/llama.cpp>
- [38] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.
- [39] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
- [40] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei *et al.*, "Qwen2. 5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.